

开发 Python

Data type

- String data: xxx via `"" / "` `print("hello") / ('hello') >> hello`
- Float data: `-x.x ~ x.x` `print(1.1+2.9) >> 4.0` 保留小数: `print(1/4) >> 0.25` | 四舍五入: `print(1//4) >> 0`
- Integer data: `-x ~ x` `print(1+1) >> 2`
- Boolean data: `true / false` `print(10<5) >> False`
- List data: can change element `print(["a","b"]) >> ['a','b']`
- Turple data: cannot change element > forensic analysis `print("a","b") >> 'a','b'`
- Dictionary data: a key mapped to a value `{1:"East", 2:"West"}`
- Set data: unique values `{"wer","erw"}`
- variables `username = "alice"`
`print(username) >> alice`

Conditional

- `==`: equal to
`!=`: not equal to
 - if failed_attempt >= 5 :
`print("Account locked")`
 - `os == "Linux 7"`
`if os == "Linux 6" :`
`print("Updated needed")`
`elif os == "Linux 5" :`
`print("Obselated")`
`else:`
`print("No updates needed")`
- in: among
 - approved_list = ["a", "b"]
username = "a"
`if username in approved_list :`
`print("user has access")`
- and / or / not
 - `if not(os_version < 7) :`
`print("No updates needed")`

Iterative/Loop

- for: repeat
 - `for i in [1,2,3] :`
`print(i) >> 1 2 3`
 - `range (start=0, end, increment=1 by default)`
 - `for i in range(1,4,2) :`
`print(i) >> 1 3`
 - `for i in range(2) :`
`print("loop") >> loop loop`
- while: repeat with condition
 - `max_attempts = 5`
`i = 1`
`while i < max_attemps :`
`print("can access")`
`i = i+1`
`print("reach maximum") >> can access*4 reach maximum`
- break / continue: stop / skip
 - `os_ver = ["1", "2", "3"]`
`for os in os_ver :`
`if os == "2" :`
`break / continue`
`print(os) >> 1 / 1 3`
- stop an infinity loop by CTRL-C / CTRL-Z

Function

- def: define a function
 - def function header():
function body
 - `def list_to_string():`
`username_list = ["a","b"]`
`for i in username_list:`
`print (i)`
`list_to_string() >>`
`a`
`b`
 - +parameters, arguments
 - `def greet_employee(first_name,last_name):`
`print("Welcome", first_name,last_name)`
`greet_employee("Sue","Sun") >>`
`Welcome Sue Sun`
 - +return
 - `def calculate_fails(total,failed):`
`failed_percentage = failed / total`
`return failed_percentage`
`calculate_fails(4,2) >>`
`0.5`
- build-in
 - `username_list = ["b","a"]`
`print(sorted(username_list)) >>`
`['a', 'b']`

Library

- a collection of modules — Module: a python file contains additional functions, variables, classes, runnable code
- Python Standard Library: re - search pattern for logs, csv, glob/os - CLI, time/datetime, statistics...
 - `import statistics / from statistics import mean`
`failed_attempts = [20, 10, 30]`
`mean_failed_attempts = statistics.mean(failed attempts)`
`print(mean_failed_attempts)`
- External libraries: Beautiful Soup - parse html files, Numpy - math
 - `%pip install numpy`
`import numpy`

Style Guide

- a manual informs writing, formatting, document design
- PEP 8 Style Guide: Python Enhancement Proposal

String

- `str()`: convert input to string `""+"":` combine two strings `len()`: calculate length of string
- `"Hello".upper() / .lower() >> HELLO / hello`
- indice: start from 0 / end with -1 - `"Hello"[4/-4] >> o` `slice: multiple but not include last indice - "Hello"[1:4] >> ell`
- `.index()`: find first indice - `"Hello".index("l") >> 2`

List

- `list1 + list2`: combine two lists
- `list = ["a","b"]` — indice: `list[1] >> b`
 - change: `list[1] = "c" >> ['a','c']`
 - add: `list.insert(1,"c") >> ['a','c','b']` — `list.append("c") >> ['a','b','c']`
 - delete: `list.remove("b") >> ['a']`
- `IP = ["191.1.1.1","192.2.2.2","193.3.3.3"]`
`network = [] #create an empty list`
`for address in IP: #loop`
`network.append(address[0:3])`
`print(network) >> ['191','192','193']`

Regex

- Regular expression: re function to search with pattern instead of specific value
- `re.findall()`: return matches
 - `import re`
`email_address = ["x","x"]`
`re.findall("\w+@\w+\.com",email_address)) >> ['test@email.com']`
- +: one or more `"a+", "a,aaa,aaa" >> ['a','aa','aaa']`
- \w: one but no symbol `\w, a1 >> ['a','1']`
- .: any character
 - `\d`: one digit `\d, a11b2 >> ['1','1','2']`
 - `\d+, a11b2 >> ['11','2']`
 - `\d{2}, a11 b2 >> ['11']`
 - `\d{1,2}, a11 b2 c333 >> ['11','2']`
range
 - `\d*, a12b2 >> ['','12','','2','']`
#include zero
 - `\s`: one space `"\, ", "`

File

- with: handle error and manage external resources
 - `with open("/home/x/x.csv","r") as file: #open a file to read and store file in variable`
 - `file.read()` #convert file to strings to read
`a, b`
 - `with open("/home/x/x.csv","w") as file: #open a file to write / create a new file`
 - `new = ["a"]`
`with open("x.csv","a") as file: #open a file to append`
`file.write(new)` #write "a" to file
- parsing: convert to readable format
 - `file.split()` #convert string to list
`['a','b']`
 - `updated_file = "\n".join(updated_file) #convert list to string so as to save as file`
`with open("x.csv","w") as file:`
`file.write(updated file)`
- counter
 - `usernames = login_attempts.split()`
`def login_check(list, user):`
`counter = 0`
`for i in list:`
`if i == user:`
`counter = counter + 1`
`if counter >= 3:`
`return "Account locked"`
`else:`
`return "can log in"`
`login_check(usernames, "sue")`

Debug

- Error
 - syntax error: e.g. miss `":`
 - logic error: check by dedugger / add `print()` to run codes in segments
 - exception: `NameError, IndexError, TypeError, FileNotFoundError`, e.g. value not defined, `"Hello"[100]`, divide 0
- Debugger
 - IDE: a software for writing code to assist editing and debugging
 - e.g. Visual Studio Code, JetBrains
 - + AI: e.g. Gemini Code Assit
 - Integrated Development Environment